OS-ORANGE PROBLEM

NAME:SANJEEV MURTHY B

SRN:PES1UG24CS612

Phase 1: Object Storage Foundation

1A:

```
sanjeev@819889:~/PES1UG24CS612-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
sanjeev@819889:~/PES1UG24CS612-pes-vcs$
```

1B:

```
sanjeev@819889:~/PES1UG24CS612-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
sanjeev@819889:~/PES1UG24CS612-pes-vcs$
```

Phase 2: Tree Objects

2A:

```
sanjeev@819889:~/PES1UG24CS612-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
sanjeev@819889:~/PES1UG24CS612-pes-vcs$
```

2B:

```
sanjeev@819889:~/PES1UG24CS612-pes-vcs$ find .pes/objects -type f
.pes/objects/8e/69de63b8e0d86b6775d19fedc83c030a4dbffee623f163e8e03a782c92d5e8
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/dd/7493dd11a4e1855394b78721d8c44a16ff6db5efb484e209c638552d65d941
sanjeev@819889:~/PES1UG24CS612-pes-vcs$
```

```
sanjeev@819889:~/PES1UG24CS612-pes-vcs$ hexdump -C $(find .pes/objects -type f | head -n 1)
00000000  63 6f 6d 6d 69 74 20 35  30 00 74 72 65 65 20 dd  |commit 50.tree .|
00000010  74 93 dd 11 a4 e1 85 53  94 b7 87 21 d8 c4 4a 16  |t......S...!..J.|
00000020  ff 6d b5 ef b4 84 e2 09  c6 38 55 2d 65 d9 41 74  |.m.......8U-e.At|
00000030  72 65 65 20 7f 0a 0a 74  65 73 74 0a              |ree ...test.|
0000003c
sanjeev@819889:~/PES1UG24CS612-pes-vcs$
```

Phase 3: The Index (Staging Area)

3A:

```
sanjeev@819889:~/PES1UG24CS612-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  a.txt
  untracked:  tree.h
  untracked:  test_tree.c
  untracked:  commit.c
  untracked:  tree.c
  untracked:  test_objects.c
  untracked:  test_objects
  untracked:  README.md
  untracked:  commit.h
  untracked:  index.h
  untracked:  index.c
  untracked:  .gitignore
  untracked:  test_tree
  untracked:  test_sequence.sh
  untracked:  object.c
  untracked:  Makefile
  untracked:  pes.h
  untracked:  pes.c
```

3B:

```
sanjeev@819889:~/PES1UG24CS612-pes-vcs$ cat .pes/index
100664 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776342755 6 file1.txt
100664 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776342755 6 file2.txt
sanjeev@819889:~/PES1UG24CS612-pes-vcs$
```

Phase 4: Commits and History

4A:

```
sanjeev@819889:~/PES1UG24CS612-pes-vcs$ ./pes log
commit 73af71aa5c1d4e8b8e6a34643dba8f9885dd7f5c9726e7c5508254fa90531d08
Author: PES User <pes@localhost>
Date:   1776343384

    Add farewell

commit 8e0e442973e0a344d01f24c4225cc7a5136c6ac31005c2a944268625dbbfb185
Author: PES User <pes@localhost>
Date:   1776343384

    Add worldell

commit 49658c22b8c6f1fbb841ac45da583396e9e9b4f48dd5664a47198609fa49185d
Author: PES User <pes@localhost>
Date:   1776343384

    Initial commit

commit 0b65950fdc9a33461d378437af22065043c96aab66537707ad99ab39497ea920
Author: PES User <pes@localhost>
Date:   1776343304

    first330a

sanjeev@819889:~/PES1UG24CS612-pes-vcs$
```

4B:

```
sanjeev@819889:~/PES1UG24CS612-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/0b/65950fdc9a33461d378437af22065043c96aab66537707ad99ab39497ea920
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/49/658c22b8c6f1fbb841ac45da583396e9e9b4f48dd5664a47198609fa49185d
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/73/af71aa5c1d4e8b8e6a34643dba8f9885dd7f5c9726e7c5508254fa90531d08
.pes/objects/8e/0e442973e0a344d01f24c4225cc7a5136c6ac31005c2a944268625dbbfb185
.pes/objects/dd/7493dd11a4e1855394b78721d8c44a16ff6db5efb484e209c638552d65d941
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
sanjeev@819889:~/PES1UG24CS612-pes-vcs$
```

4C:

```
sanjeev@819889:~/PES1UG24CS612-pes-vcs$ cat .pes/refs/heads/main
73af71aa5c1d4e8b8e6a34643dba8f9885dd7f5c9726e7c5508254fa90531d08
sanjeev@819889:~/PES1UG24CS612-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
sanjeev@819889:~/PES1UG24CS612-pes-vcs$
```

QUESTIONS:

Q5.1: Branching and Checkout

A branch in Git (and PES) is a file in .pes/refs/heads/ that stores a commit hash. To implement pes checkout <branch>, the following steps are required:

1. The .pes/HEAD file must be updated to point to the selected branch, i.e., it should contain ref: refs/heads/<branch>.
2. The commit hash stored in .pes/refs/heads/<branch> must be read.
3. The corresponding commit object should be loaded, and its tree should be retrieved.
4. The working directory must be updated to match this tree. This involves deleting or modifying existing tracked files and recreating files from the tree by reading blob objects.
5. The index must also be updated to match the new tree state.

This operation is complex because it involves safely modifying the working directory, handling nested directory structures, and ensuring that uncommitted changes are not lost. It requires careful state management and file reconstruction.

Q5.2: Detecting Dirty Working Directory Conflicts

To detect conflicts when switching branches, the system must compare three versions of each tracked file: the working directory, the index, and the target branch.

1. First, check if the working directory file differs from the index by comparing metadata such as file size and modification time.
2. If the file is modified, compare the index version with the version in the target branch.
3. If both differ, it indicates a conflict.

A conflict occurs when:

- The working directory file is different from the index (uncommitted changes), and
- The index version is different from the target branch version.

In such cases, the checkout operation must be aborted to prevent data loss.

Q5.3: Detached HEAD

A detached HEAD state occurs when .pes/HEAD contains a commit hash directly instead of a reference to a branch.

In this state, new commits can still be created, but they are not associated with any branch. As a result, these commits become unreachable once the user switches to another branch, and they may eventually be removed during garbage collection.

To recover such commits, the user can create a new branch pointing to the commit using a command like:
git branch <new_branch> <commit_hash>
or
git checkout -b <new_branch>

This ensures that the commits become reachable again.

Q6.1: Garbage Collection and Reachability

Over time, the object store accumulates unreachable objects that are not referenced by any branch.

To remove them, a mark-and-sweep algorithm can be used:

1. Start from all branch heads in .pes/refs/heads/.
2. Traverse each commit recursively, marking all reachable objects (commits, trees, and blobs).
3. Store all reachable object hashes in a hash set for efficient lookup.
4. Scan the entire .pes/objects/ directory.
5. Delete any object whose hash is not present in the reachable set.

A hash set is used because it provides constant-time lookup and is efficient for large repositories.

For a repository with 100,000 commits and 50 branches, approximately 100,000 commit objects, 100,000 tree objects, and several hundred thousand blob objects may need to be visited. This results in around 500,000 to 700,000 objects in total.

Q6.2: Garbage Collection Race Condition

Running garbage collection concurrently with a commit operation is dangerous due to race conditions.

For example:

1. A commit operation creates new objects (blobs, trees, commit) but has not yet updated the branch reference.
2. At the same time, garbage collection scans the object store.
3. Since the new objects are not yet referenced by any branch, GC considers them unreachable and deletes them.
4. The commit operation then tries to update the branch to point to a commit that references deleted objects, leading to repository corruption.

Git avoids this issue using several mechanisms:

- Locking to prevent concurrent modifications during critical operations.
- Atomic updates to ensure references are updated only after all objects are safely written.
- Temporary references to protect newly created objects.
- A grace period before deleting unreachable objects.

These mechanisms ensure that objects are not prematurely deleted and maintain repository consistency.